Microsoft Defender for Endpoint for macOS — Deployment & Onboarding via Intune

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune) 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 5

Outcome . . . 6

1. Solution Overview — Microsoft Defender for Endpoint for macOS: Deployment & Onboarding via Intune 6

What this solution does . . . 6

The macOS deployment model (why order matters) . . . 6

The three Intune delivery surfaces on macOS . . . 7

Onboard-then-verify: the health model . . . 7

Benefits . . . 7

Limitations (be honest with buyers) . . . 8

The deployment model . . . 8

2. Requirements Guide — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune) 8

2.1 Microsoft Defender for Endpoint licensing . . . 9

2.2 Apple MDM Push (APNs) certificate . . . 9

2.3 macOS enrollment configured . . . 9

2.4 Supported macOS version . . . 9

2.5 The MDE onboarding package (from the Microsoft Defender portal) . . . 10

2.6 Microsoft Defender extension identifiers and code requirements . . . 10

2.7 Pilot and production device groups . . . 10

2.8 Network access to Microsoft Defender endpoints . . . 10

2.9 (Optional) Local admin / Terminal access for on-device validation . . . 10

Checklist . . . 11

Global Requirements — Intune Solutions (category-level) 11

G1. Microsoft Entra ID tenant . . . 11

G2. Microsoft Intune subscription . . . 11

G3. Administrative roles . . . 11

G4. Device enrollment in Intune . . . 12

G5. Pilot security group . . . 12

G6. Supported Windows baseline . . . 12

G7. Network access to Microsoft cloud endpoints . . . 12

3. Implementation Guide — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune) 12
Step 1 — Stage the deployment (Assess) . . . 13
Step 2 — Confirm Apple management prerequisites . . . 13
Step 3 — Create the device groups . . . 13
Step 4 — Build the System Extensions profile (capability profile 1 of 5) . . . 13
Step 5 — Build the Full Disk Access (PPPC / TCC) profile (capability profile 2 of 5) . . . 14
Step 6 — Build the Network filter profile (capability profile 3 of 5) . . . 14
Step 7 — Build the Background services and Notifications profiles (capability profiles 4 and 5 of 5) 14
Step 8 — Deploy the Microsoft Defender app . . . 14
Step 9 — Deploy the onboarding profile (after the app) . . . 15
Step 10 — Validate MDE health on the pilot, with pilot gate and rollback note . . . 15
Step 11 — Expand to production . . . 16
5. Testing Guide — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune) 16
Test 1 — System Extensions profile applied to the pilot . . . 16
Test 2 — Full Disk Access (PPPC/TCC) granted . . . 16
Test 3 — Network filter profile applied . . . 16
Test 4 — Background services and Notifications profiles applied . . . 17
Test 5 — Capability-before-onboarding gate . . . 17
Test 6 — Microsoft Defender app installed and onboarding profile applied . . . 17
Test 7 — Agent health on the device (the real finish line) . . . 17
Test 8 — Cloud connectivity and portal activity . . . 17
Test 9 — Detection test (EDR sensor working) . . . 17
Test 10 — Fleet posture and drift report . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune) 18
A. Onboarded in the portal but not actually protecting (the classic macOS failure) . . . 18
B. Full Disk Access not granted . . . 18
C. Network protection / web filtering inert . . . 19
D. Agent installed but never onboards / stays inactive in the portal . . . 19
E. User prompts appearing (should have been silent) . . . 19
F. App will not install . . . 19
G. Ordering mistake (onboarding pushed first) . . . 20
H. Graph scripts fail to connect . . . 20
I. Diagnostics quick reference . . . 20

J. When to escalate 20
7. Best Practices — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune) 20
8. Deliverables Summary — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune) 22
Documentation 22
PowerShell scripts 22
Object naming convention (used across docs and scripts) 23
What the buyer receives 23
Outcome 23
Appendix A — PowerShell Scripts 25

Executive Summary — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune)

Deploying **Microsoft Defender for Endpoint (MDE) on macOS** is not the one-click app push it looks like. macOS deliberately blocks security software from loading its system extensions, reading the whole disk, and filtering network traffic until an administrator has *explicitly approved* each of those capabilities. Push the installer and the onboarding package before those approvals are in place and the agent installs but sits **silently degraded** — extensions not running, real-time protection off, no EDR sensor — while the portal shows a reassuring "onboarded" tick. Whole Mac fleets have been "onboarded" this way and protected nothing. This package deploys and onboards MDE for macOS through **Microsoft Intune** the way macOS actually requires: the five required approval profiles **first**, then the app and the onboarding package, then a health check that proves the agent is genuinely working — deployed pilot-first and rollback-ready.

This is a complete, **deploy-it-today** package written for an administrator with no prior MDE-for-macOS experience.

What you get

- A plain-language **Solution Overview** of the MDE-for-macOS deployment model — system extensions,

Full Disk Access (PPPC/TCC), the network filter, background service management, notifications, and the onboarding blob delivered via Intune — and exactly why the order matters.

- A **Requirements Guide** with why / verify / implement-if-missing for every prerequisite — MDE

licensing, the Apple MDM Push (APNs) certificate, macOS enrollment, the onboarding package, and the Microsoft-published extension identifiers — plus a consistent object-naming convention.

- An **Implementation Guide** of precise, click-by-click Intune steps: build the five named capability

profiles first, gate on them applying, deploy the Microsoft Defender app, deploy the onboarding profile, validate `mdatp` health on a pilot, and expand — with a rollback note.

- Five **PowerShell scripts** for readiness discovery, per-device validation, fleet posture and drift

reporting, safe remediation, and safe scope widening.

- **Testing** and **Troubleshooting** guides — including the classic "onboarded but not protecting"

failure — plus **Best Practices** centered on the extension-approval order.

What this delivers

A **correctly onboarded, genuinely healthy** Microsoft Defender for Endpoint deployment across the Mac fleet: the system-extension, Full Disk Access, network-filter, background-service, and notification approvals in place *before* onboarding, the agent proven healthy (extensions running, real-time protection on, active in the Microsoft Defender portal), and a documented rollback.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with the invariant rule: **profiles first, onboarding second, validation third.**

Outcome

Every Mac protected by Defender for Endpoint — the required approvals in the right order, a clean onboarding, and a proven-healthy agent — deployable by a single administrator and validated on a pilot before fleet rollout.

1. Solution Overview — Microsoft Defender for Endpoint for macOS: Deployment & Onboarding via Intune

What this solution does

Microsoft Defender for Endpoint (MDE) protects macOS with anti-malware, endpoint detection and response (EDR), network protection, and vulnerability management. But on a Mac, MDE is **not** a simple app push. macOS deliberately blocks security software from loading kernel/system extensions, reading the full disk, or filtering network traffic until an administrator has explicitly approved each of those capabilities. If you push the MDE installer and the onboarding package **before** those approvals are in place, the agent installs but sits **silently degraded** — extensions not running, real-time protection off, no EDR sensor — and it will *look* onboarded in the portal while protecting nothing. This solution delivers the required Apple approval profiles **first**, then the onboarding configuration, then validates that the agent is genuinely healthy — deployed pilot-first and rollback-ready through **Microsoft Intune**.

The macOS deployment model (why order matters)

macOS enforces user/administrator consent for exactly the capabilities MDE needs. When those consent gates are pre-approved by an MDM (Intune) configuration profile, the user sees no prompts and the agent works immediately. When they are missing, the user is either prompted (and usually clicks the wrong thing) or silently blocked. The rule for this entire solution is therefore: **profiles first, onboarding second, validation third.** The five capability profiles are:

1. **System extensions** — MDE for macOS runs as *system extensions* (the modern replacement for kernel extensions). A **System Extensions** configuration profile pre-approves the Microsoft Defender team identifier and the specific extension bundle identifiers so they load without a user prompt. Without this, the endpoint-security and network extensions never activate.

1. **Full Disk Access (Privacy Preferences Policy Control / PPPC / TCC)** — macOS's Transparency, Consent and Control (TCC) framework blocks apps from reading protected files unless granted Full Disk Access. MDE needs Full Disk Access to scan the whole file system and to run its endpoint security extension. A **PPPC** profile (also called a *Privacy Preferences Policy Control* payload) grants it at the MDM level so no user prompt is required.

1. **Network filter (content filter)** — MDE's network protection and web content filtering run as a *network extension* that inspects traffic. A **Web content filter** configuration profile pre-approves that network extension so traffic filtering can attach. Without it, network protection is inert.

1. **Background service management (login items / managed background tasks)** — recent macOS versions

let users disable an app's background items, which would stop the MDE daemons. A **Managed login items / background service management** profile keeps the Microsoft Defender background services from being turned off by the user.

1. **Notifications** — so the user actually sees MDE alerts (threat found, action needed) instead of

silent notifications, a **Notifications** configuration profile pre-approves Microsoft Defender to post notifications.

Only **after** those are assigned and applying do you deliver the **onboarding package** — the tenant-specific blob (`WindowsDefenderATPOnboarding.plist`, delivered inside a `.mobileconfig` / custom profile) that tells the installed agent which Microsoft Defender tenant to report to — and the **MDE app** itself (via the Microsoft Defender app in Intune, or a packaged installer).

The three Intune delivery surfaces on macOS

macOS is configured by Apple **configuration profiles** (`.mobileconfig` payloads) and by apps — not by Windows constructs. This solution uses three Intune surfaces:

1. **Templates** → **configuration profiles** — the System Extensions, PPPC/Full Disk Access, Web content

filter, Notifications, and Managed login items payloads are delivered as Intune macOS configuration profiles (Settings catalog where the payload is exposed, or Templates → Custom with a `.mobileconfig` where it is not).

1. **Custom** `.mobileconfig` **(onboarding)** — the MDE onboarding package is delivered as a **custom**

macOS configuration profile carrying the onboarding payload.

1. **App deployment** — the **Microsoft Defender** app is deployed via Intune (Apps → macOS →

Microsoft Defender, or a packaged `.pkg`/DMG app).

Onboard-then-verify: the health model

Getting a green "onboarded" tick in the Microsoft Defender portal is not the same as being protected. This solution treats **health** as the real finish line: after onboarding you confirm the system and network extensions are **running**, real-time protection is **on**, the device appears in the Microsoft Defender portal (`https://security.microsoft.com`) as **onboarded and active**, and the built-in health check (`mdatp health`) reports healthy. The included scripts read the Intune side of this (profile apply state, app install state, device enrollment) so you can prove the management-plane prerequisites are in place, and the guides show the on-device `mdatp` commands that prove the agent itself is healthy.

Benefits

• **Onboarded correctly, not just installed** — the required Apple approvals land *before* onboarding,

so the agent is genuinely protecting the Mac rather than silently degraded.

• **No end-user prompts** — every consent gate (extensions, Full Disk Access, network filter,

background items, notifications) is pre-approved by MDM, so users aren't asked to make security decisions.

• **Provable health** — validation confirms extensions running, real-time protection on, and the device active in the Microsoft Defender portal.

• **Safe rollout** — piloted on a small group and validated before fleet-wide, with a documented rollback because a bad onboarding or a missing approval is easy to correct when caught early.

Limitations (be honest with buyers)

• The exact Microsoft Defender **team identifier**, **extension bundle identifiers**, **code requirement strings**, **and the** onboarding package** are Microsoft-published values — this solution gives the correct procedure and object names; where a specific identifier is required it is clearly marked as *obtain from the current Microsoft documentation / your onboarding download*, not invented.

• MDE licensing and a working macOS enrollment are prerequisites, not part of this solution.

• Some macOS versions surface these payloads differently (Settings catalog vs custom `.mobileconfig`); the implementation flags where a custom profile may be required.

The deployment model

> *Deliver the System Extensions, Full Disk Access (PPPC/TCC), Network filter, Background services, and Notifications profiles **first**; then deploy the MDE app and the onboarding package; then validate the agent is healthy — on a pilot, before you expand. The repeatable way: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune)

Verify **every** requirement before configuring. Each entry: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Object names used throughout this solution (reuse them verbatim in Intune):

• System Extensions profile: `MDE macOS – System Extensions`

• Full Disk Access (PPPC/TCC) profile: `MDE macOS – Full Disk Access`

• Network filter (web content filter) profile: `MDE macOS – Network Filter`

• Background services (managed login items) profile: `MDE macOS – Background Services`

• Notifications profile: `MDE macOS – Notifications`

• Onboarding profile (custom `.mobileconfig`): `MDE macOS – Onboarding`

• Microsoft Defender app deployment: `MDE macOS – Defender App`

- Device groups: `MDE macOS – Pilot Devices` and `MDE macOS – Production Devices`
- Tag (put in every profile/app description): `MDE-macOS`

> ***Order rule (applies to the whole solution):*** *the five capability profiles above are delivered and confirmed applying* ***before*** *the onboarding profile and app. Onboarding a Mac whose extensions, Full Disk Access, and network filter are not yet approved produces a* ***silently degraded*** *agent.*

2.1 Microsoft Defender for Endpoint licensing

Description: A Microsoft Defender for Endpoint plan that covers macOS (for example Microsoft 365 E5 / E5 Security, Microsoft Defender for Endpoint Plan 1 or Plan 2, or Microsoft 365 Business Premium where eligible). **Why required:** Without an MDE license the onboarding package will not be available and the device cannot report to the Microsoft Defender portal. **How to verify:** `https://security.microsoft.com` loads and **Settings** → **Endpoints** → **Onboarding** offers **macOS** as an operating system; Microsoft 365 admin center → **Billing** → **Licenses** shows an MDE-bearing plan. **How to implement if missing:** Assign an MDE-bearing license, then confirm macOS onboarding is available in the Microsoft Defender portal.

2.2 Apple MDM Push (APNs) certificate

Description: A valid Apple Push Notification service (APNs) certificate configured in Intune. **Why required:** Apple devices (including Macs) cannot be managed by Intune without an APNs certificate — no configuration profiles, apps, or onboarding payloads reach the Mac until it exists. **How to verify:** Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate**; confirm present and **not expired**. **How to implement if missing:** Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate** → **download the CSR**, create the certificate at `https://identity.apple.com` with a long-lived Apple ID, upload it back, and record the renewal date.

2.3 macOS enrollment configured

Description: A working macOS enrollment method so target Macs are enrolled and manageable — Automated Device Enrollment (ADE/DEP), Account-driven enrollment, or Company Portal device enrollment. **Why required:** Profiles, apps, and the onboarding payload apply only to enrolled Macs. Note that **supervised** Macs (ADE) allow the smoothest silent approval of extensions and network filters. **How to verify:** Intune → **Devices** → **macOS** → **macOS devices**; the Mac appears as managed. Run `MACMDE-Discovery.ps1` for tenant/device readiness signals. **How to implement if missing:** Configure **Automated Device Enrollment** (Apple Business/School Manager token + an enrollment profile) for corporate Macs, or **Account-driven**/Company Portal enrollment for user-owned Macs; then enrol the devices.

2.4 Supported macOS version

Description: A currently supported macOS major version on the target Macs (MDE for macOS supports the current and recent macOS releases). **Why required:** MDE's system extensions, network extension, and TCC behavior depend on a supported macOS version; older, unsupported versions may fail to load extensions or expose payloads differently. **How to verify:** On a Mac, **Apple menu** → **About This Mac**; or Intune → device → **Hardware**. Run `MACMDE-Discovery.ps1` for the fleet macOS-version spread. **How to implement if missing:** Update the Macs to a supported macOS version before onboarding.

2.5 The MDE onboarding package (from the Microsoft Defender portal)

Description: The macOS onboarding package downloaded from the Microsoft Defender portal — it contains the tenant-specific `WindowsDefenderATPOnboarding.plist` / onboarding `.mobileconfig`. **Why required:** This blob tells the installed MDE agent which Microsoft Defender tenant to report to. It is tenant-specific and is the authoritative onboarding source — do not hand-craft it. **How to verify:** `https://security.microsoft.com` → **Settings** → **Endpoints** → **Onboarding** → operating system **macOS** → deployment method **Mobile Device Management / Microsoft Intune** → **Download onboarding package**; you have the extracted onboarding profile. **How to implement if missing:** Download the onboarding package from the Microsoft Defender portal as above and extract the onboarding configuration profile for use in `MDE macOS – Onboarding` (Implementation Step 8).

2.6 Microsoft Defender extension identifiers and code requirements

Description: The Microsoft Defender **team identifier**, **system extension bundle identifiers**, **network extension identifier**, and the **PPPC code requirement string** used to pre-approve the extensions and Full Disk Access. **Why required:** The System Extensions, Network filter, and Full Disk Access profiles must name the exact Microsoft-published identifiers or macOS will not silently approve the extensions. **How to verify:** You have the current identifier/code-requirement values from Microsoft's macOS deployment documentation (or the Settings catalog exposes a Microsoft Defender preset that carries them). **How to implement if missing:** Obtain the current identifiers and code requirement from Microsoft's official *Deploy Microsoft Defender for Endpoint on macOS with Intune* documentation, or use the Intune Settings-catalog Microsoft Defender presets where available. *(These are Microsoft-published values — use the current ones; this solution does not invent them.)*

2.7 Pilot and production device groups

Description: Two Entra ID security groups: `MDE macOS – Pilot Devices` (a small representative sample of Macs) and `MDE macOS – Production Devices` (the rest of the Mac fleet). **Why required:** Onboarding is piloted before fleet rollout so a missing approval or a degraded agent is caught on a few Macs, not the whole fleet. **How to verify:** Entra → **Groups**; both groups exist. **How to implement if missing:** Entra → **Groups** → **New group** → **Security**; add representative pilot Macs to one and the remaining Macs to the other.

2.8 Network access to Microsoft Defender endpoints

Description: Outbound HTTPS from the Macs to the Microsoft Defender for Endpoint service URLs (in addition to the general Microsoft cloud endpoints in the global requirements). **Why required:** The onboarded agent must reach the MDE cloud to report health, telemetry, and EDR data; blocked endpoints leave the agent installed but inactive in the portal. **How to verify:** From a Mac, the MDE connectivity test passes (`mdatp connectivity test`), or the device reports **active** in the Microsoft Defender portal after onboarding. **How to implement if missing:** Allow the Microsoft Defender for Endpoint published service URLs through the proxy/firewall (per Microsoft's documented MDE endpoint list).

2.9 (Optional) Local admin / Terminal access for on-device validation

Description: The ability to run `mdatp` commands in Terminal on a pilot Mac (for example `mdatp health`). **Why required:** The definitive health proof (extensions running, real-time protection on) is read on the device with the `mdatp` CLI; the Intune-side scripts prove the management prerequisites but not the live agent state. **How to verify:** On a pilot Mac, `mdatp health` returns output. **How to implement if missing:** Ensure a pilot operator can open Terminal on a pilot Mac to run the validation commands in the Testing Guide.

Checklist

- Microsoft Defender for Endpoint licensing assigned; macOS onboarding available in the Defender portal
- Apple MDM Push (APNs) certificate present and not expired
- macOS enrollment configured (ADE / Account-driven / Company Portal); Macs enrolled — `MACMDE-Discovery.ps1`
- Supported macOS version on target Macs
- MDE macOS onboarding package downloaded from the Defender portal
- Microsoft Defender extension identifiers / PPPC code requirement obtained (Microsoft-published)
- `MDE macOS – Pilot Devices` and `MDE macOS – Production Devices` groups created
- Network access to Microsoft Defender service endpoints allowed
- (Optional) Terminal access on a pilot Mac for `mdatp` health validation

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune)

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`) and the **Microsoft Defender portal** (`https://security.microsoft.com`). This delivers the five required Apple approval profiles **first**, then the Microsoft Defender app and the onboarding package, then validates that the agent is genuinely healthy — proven on a pilot, then expanded fleet-wide.

> ***Model:*** *Assess → Configure → Pilot → Validate → Expand. A small* `MDE macOS – Pilot Devices` *group validates onboarding before the* `MDE macOS – Production Devices` *fleet receives it.* ***Do not deploy the onboarding profile or the app before the five capability profiles are applying — onboarding a Mac without them produces a silently degraded agent (Step 10 gate).***

Source note: *The Microsoft Defender **team identifier**, **extension bundle identifiers**, **network extension identifier**, **PPPC code requirement**, and the **onboarding package** are Microsoft-published, tenant-specific values. Obtain them from the current Microsoft *Deploy Microsoft Defender for Endpoint on macOS with Intune* documentation and your onboarding download — this guide gives the exact procedure and object names; it does not invent identifier values.*

Step 1 — Stage the deployment (Assess)

1. In the **Microsoft Defender portal** (`https://security.microsoft.com`) → **Settings** → **Endpoints** → **Onboarding**, select operating system **macOS** and deployment method **Mobile Device Management / Microsoft Intune**, and **download the onboarding package** (Requirement 2.5). Extract the onboarding configuration profile.

2. Obtain the current Microsoft Defender **extension identifiers** and **PPPC code requirement** (Requirement 2.6), or confirm the Intune Settings catalog exposes the Microsoft Defender presets.

3. Confirm you will tag every object with `MDE-macOS` in its description for auditability.

Step 2 — Confirm Apple management prerequisites

1. Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate**; confirm present and not expired (Requirement 2.2).

2. Confirm your target Macs are enrolled: Intune → **Devices** → **macOS** → **macOS devices** (Requirement 2.3). Supervised (ADE) Macs approve extensions and network filters most smoothly.

3. Run `MACMDE-Discovery.ps1` for a readiness snapshot (APNs, enrolled Macs, macOS versions, any existing Defender/security profiles that could conflict).

Step 3 — Create the device groups

1. Entra → **Groups** → **New group** → **Security** → name `MDE macOS – Pilot Devices` → add a few representative Macs. **Create**.

2. Create `MDE macOS – Production Devices` for the rest of the fleet. **Create**.

Step 4 — Build the System Extensions profile (capability profile 1 of 5)

1. Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** → **Platform: macOS**.

2. **Profile type: Settings catalog** (search for **System Extensions**), or **Templates** → **Extensions** if your tenant surfaces the template. **Create**.

3. **Name:** `MDE macOS – System Extensions`. **Description:** include the tag `MDE-macOS`. **Next**.

4. Add an **allowed system extensions** entry: set the **team identifier** to the Microsoft Defender team identifier and add the **allowed system extension types / bundle identifiers** for the Microsoft Defender endpoint-security and network extensions (Requirement 2.6). This pre-approves the extensions so they load without a user prompt.

5. **Next** → **Assignments** → **Add groups** → `MDE macOS – Pilot Devices`. **Next** → **Create**.

Step 5 — Build the Full Disk Access (PPPC / TCC) profile (capability profile 2 of 5)

1. Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** → **Platform: macOS** → **Templates** → **Privacy Preferences Policy Control** (or **Settings catalog** → **Privacy**). **Create**.

2. **Name:** `MDE macOS – Full Disk Access`. **Description:** tag `MDE-macOS`. **Next**.

3. Add a PPPC entry that grants **SystemPolicyAllFiles (Full Disk Access) = Allow** to the Microsoft Defender app/extension, identified by its bundle identifier and **code requirement** string (Requirement 2.6). This grants MDE the whole-disk read access it needs at the MDM level, so no user prompt is required.

4. **Next** → **Assignments** → **Add groups** → `MDE macOS – Pilot Devices`. **Next** → **Create**.

Step 6 — Build the Network filter profile (capability profile 3 of 5)

1. Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** → **Platform: macOS** → **Templates** → **Web content filter** (or **Settings catalog** → **Content filter**). **Create**.

2. **Name:** `MDE macOS – Network Filter`. **Description:** tag `MDE-macOS`. **Next**.

3. Configure a **plug-in content filter** that names the Microsoft Defender **network extension** identifier and filter data-provider bundle identifier (Requirement 2.6). This pre-approves the network extension so MDE network protection/web filtering can attach.

4. **Next** → **Assignments** → **Add groups** → `MDE macOS – Pilot Devices`. **Next** → **Create**.

Step 7 — Build the Background services and Notifications profiles (capability profiles 4 and 5 of 5)

1. **Background services:** Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** → **Platform: macOS** → **Settings catalog** → **search Managed Login Items / Background Task Management**. **Name:** `MDE macOS – Background Services`. Add a **managed login item / allowed background service** rule for the Microsoft Defender team identifier so the user cannot disable MDE's background daemons. **Assign** to `MDE macOS – Pilot Devices`. **Create**.

2. **Notifications:** Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** → **Platform: macOS** → **Templates** → **Notifications** (or **Settings catalog** → **Notifications**). **Name:** `MDE macOS – Notifications`. Add a notification-settings entry allowing the Microsoft Defender bundle identifier to post notifications (alerts enabled, banners/lock-screen as desired). **Assign** to `MDE macOS – Pilot Devices`. **Create**.

> ***Gate before onboarding:*** *confirm all five profiles above show* ***Succeeded*** *on a pilot Mac (Intune → each profile →* ***Device status****) before proceeding. If any is not applying, fix it now — do* ***not*** *onboard first.*

Step 8 — Deploy the Microsoft Defender app

1. Intune → **Apps** → **macOS** → **Add**. Choose **Microsoft Defender** (the built-in app type) where available, or **macOS app (PKG/DMG)** with a packaged Defender installer.

2. **Name:** `MDE macOS – Defender App`. **Description:** tag `MDE-macOS`.

3. **Assignments** → **Required** → **Add group** → `MDE macOS – Pilot Devices`. **Next** → **Create**. The app installs on the pilot Macs (with the extensions already approved from Steps 4–7, it activates cleanly).

Step 9 — Deploy the onboarding profile (after the app)

1. Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** → **Platform: macOS** → **Templates** → **Custom**. **Create**.

2. **Name:** `MDE macOS – Onboarding`. **Description:** tag `MDE-macOS`.

3. Provide a **deployment channel = Device channel** and **upload the onboarding configuration profile** extracted from the Defender portal download (Requirement 2.5 / Step 1). This carries the `WindowsDefenderATPOnboarding` payload that points the agent at your tenant.

4. **Assignments** → **Add groups** → `MDE macOS – Pilot Devices`. **Next** → **Create**.

Step 10 — Validate MDE health on the pilot, with pilot gate and rollback note

1. On a pilot Mac, force a sync: **Company Portal** → **...** → **Check status** (or **System Settings** → **General** → **Device Management**), and allow the profiles, app, and onboarding to apply.

2. In Intune, confirm each of the five profiles + `MDE macOS – Onboarding` show **Succeeded** and `MDE macOS – Defender App` shows **Installed** (each object → **Device status**). Run `MACMDE-Validation.ps1 -DeviceName "MAC-001"` for the management-plane view and `MACMDE-Report.ps1` for pilot-wide state.

3. **On the Mac (Terminal), prove the agent is healthy** (this is the real finish line — the portal tick is not enough):

• `mdatp health` — confirm `healthy : true`, `real_time_protection_enabled : true`, and the licensed/onboarded fields are true.

• `systemextensionsctl list` — confirm the Microsoft Defender endpoint-security and network extensions are **activated enabled**.

• `mdatp connectivity test` — confirm connectivity to the MDE cloud passes.

• **System Settings** → **Privacy & Security** → **Full Disk Access** — confirm Microsoft Defender is granted (from the PPPC profile, no user action).

1. In the **Microsoft Defender portal** → **Assets** → **Devices**, confirm the pilot Mac appears **onboarded** with **Active** health status (allow up to ~15–30 minutes after first onboarding).

2. **Soak the pilot:** have pilot users work normally for several business days. Complete the **Testing Guide** (document 5).

3. **Gate:** do not expand until the pilot passes the Testing Guide gate AND `mdatp health` reports healthy with extensions running and real-time protection on — a green portal tick alone is **not** a pass.

4. **Rollback (if onboarding is wrong or the agent is degraded):** in Intune, **remove the assignment** of `MDE macOS – Onboarding` (and, if needed, the app) from the affected group, then sync — this offboards cleanly on the next cycle. Fix the missing approval (usually a capability profile from Steps 4–7

that was not applying), reconfirm the five profiles succeed, then reassign onboarding. Record what you rolled back and why.

Step 11 — Expand to production

1. Add `MDE macOS – Production Devices` to each of the **five capability profiles** first, and confirm they apply across a production wave (re-run `MACMDE-Report.ps1`).

2. **Only then** add `MDE macOS – Production Devices` to `MDE macOS – Defender App` and finally to `MDE macOS – Onboarding` — preserving the profiles-before-onboarding order fleet-wide.

3. Roll out in waves if the fleet is large; after each wave re-run `MACMDE-Report.ps1` and spot-check `mdatp health` on a sample. Use `MACMDE-Set-ScopeTargeting.ps1` (default `-WhatIf`) to widen the target group membership safely.

Order of operations

`Stage onboarding + identifiers → Confirm APNs/enrollment → Groups → System Extensions → Full Disk Access (PPPC) → Network Filter → Background Services + Notifications → [GATE: 5 profiles Succeeded] → Defender App → Onboarding profile → Validate mdatp health (pilot gate / rollback) → Expand (profiles first, then app, then onboarding)`

5. Testing Guide — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune)

Run after the pilot objects apply (Implementation Step 10). Tests 1–5 confirm the **capability profiles landed before onboarding**; Tests 6–9 confirm the **agent is genuinely healthy** — not just showing a green tick.

Test 1 — System Extensions profile applied to the pilot

Do: Intune → **Devices** → **macOS** → **Configuration** → `MDE macOS – System Extensions` → **Device status**. On the Mac: Terminal `systemextensionsctl list`. **Expected:** Pilot Macs show **Succeeded**; the Microsoft Defender endpoint-security and network extensions are listed **activated enabled** with no pending user approval.

Test 2 — Full Disk Access (PPPC/TCC) granted

Do: Intune → `MDE macOS – Full Disk Access` → **Device status**. On the Mac: **System Settings** → **Privacy & Security** → **Full Disk Access**. **Expected:** Profile **Succeeded**; Microsoft Defender appears **granted** Full Disk Access **without** the user having toggled it.

Test 3 — Network filter profile applied

Do: Intune → `MDE macOS – Network Filter` → **Device status**. On the Mac: **System Settings** → **General** → **VPN & Device Management** (or Network) shows the Microsoft Defender content filter present. **Expected:** Profile **Succeeded**; the Microsoft Defender network extension/content filter is attached and

enabled.

Test 4 — Background services and Notifications profiles applied

Do: Intune → `MDE macOS – Background Services` and `MDE macOS – Notifications` → **Device status**. On the Mac: the user cannot disable Microsoft Defender background items, and Defender can post notifications. **Expected:** Both profiles **Succeeded**; MDE background daemons are protected and notifications are allowed.

Test 5 — Capability-before-onboarding gate

Do: Confirm Tests 1–4 all pass **before** checking onboarding. On the Mac run nothing new — just verify the five profiles are Succeeded. **Expected:** All five capability profiles **Succeeded**. Only then is an onboarding pass meaningful.

Test 6 — Microsoft Defender app installed and onboarding profile applied

Do: Intune → `MDE macOS – Defender App` → **Device install status**; Intune → `MDE macOS – Onboarding` → **Device status**. **Expected:** App **Installed**; onboarding profile **Succeeded**.

Test 7 — Agent health on the device (the real finish line)

Do: On a pilot Mac (Terminal): `mdatp health`. **Expected:** `healthy : true`, `real_time_protection_enabled : true`, `licensed : true`, and onboarding fields true. Run `MACMDE-Validation.ps1 -DeviceName "MAC-001"` for the Intune-side confirmation.

Test 8 — Cloud connectivity and portal activity

Do: On the Mac: `mdatp connectivity test`. In the **Microsoft Defender portal** → **Assets** → **Devices**, find the pilot Mac. **Expected:** Connectivity test passes; the Mac appears **onboarded** with **Active** health status (allow ~15–30 minutes after first onboarding).

Test 9 — Detection test (EDR sensor working)

Do: On a pilot Mac, run the Microsoft-documented macOS EDR detection test (a benign test that triggers an alert), then check the Defender portal. **Expected:** An alert appears for the pilot Mac in the Defender portal, proving the EDR sensor is reporting.

Test 10 — Fleet posture and drift report

Do: Run `MACMDE-Report.ps1`. **Expected:** A per-object breakdown across the pilot (profiles Succeeded, app Installed, onboarding Succeeded); pilot Macs show no drift.

Pass/fail gate

Gate	Requirement
Capability gate	Tests 1–5 pass — all five profiles Succeeded **before** onboarding is evaluated
Pilot pass	Tests 6–9 pass; `mdatp health` reports healthy with extensions running and real-time protection on; device Active in the Defender portal
Production pass	Tests 1–8 and 10 pass for `MDE macOS – Production Devices` after each rollout wave (profiles first, then app, then onboarding)

6. Troubleshooting Guide — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune)

Use `MACMDE-Discovery.ps1` for tenant/device readiness, `MACMDE-Validation.ps1` for a single device, `MACMDE-Report.ps1` for fleet posture and drift, `MACMDE-Remediation.ps1` for safe reapply (default `-WhatIf`), and `MACMDE-Set-ScopeTargeting.ps1` when widening scope during Expand. On the Mac itself, the `mdatp` CLI is the definitive source of agent health.

A. Onboarded in the portal but not actually protecting (the classic macOS failure)

Symptom	Cause	Remediation
Portal shows the Mac onboarded, but `mdatp health` shows `real_time_protection_enabled : false` or `healthy : false`	The capability profiles were not applied **before** onboarding (extensions never approved)	Confirm all five profiles Succeeded (`MDE macOS – System Extensions`, `– Full Disk Access`, `– Network Filter`, `– Background Services`, `– Notifications`); fix whichever is not applying, sync, then re-check `mdatp health`
`systemextensionsctl list` shows the Defender extensions not enabled / pending approval	System Extensions profile missing, wrong team identifier, or Mac not supervised	Correct the team identifier / bundle identifiers in `MDE macOS – System Extensions` (Requirement 2.6); on unsupervised Macs the user may need to approve once — prefer ADE/supervised

B. Full Disk Access not granted

Symptom	Cause	Remediation
Defender shows a Full Disk Access warning, scans incomplete	PPPC profile missing, wrong bundle identifier, or wrong code requirement string	Correct `MDE macOS – Full Disk Access` (SystemPolicyAllFiles = Allow, correct bundle id + code requirement, Requirement 2.6); confirm **System Settings** → **Privacy & Security** → **Full Disk Access** shows Defender granted

C. Network protection / web filtering inert

Symptom	Cause	Remediation
Network protection not working; content filter absent under Network/Device Management	Network filter profile missing or names the wrong network-extension identifier	Correct `MDE macOS – Network Filter` with the Microsoft Defender network extension identifier; confirm the content filter is attached on the Mac

D. Agent installed but never onboards / stays inactive in the portal

Symptom	Cause	Remediation
App installed, `mdatp health` shows onboarded false	`MDE macOS – Onboarding` not assigned/applied, or wrong onboarding package (different tenant)	Confirm the onboarding profile Succeeded and uses the package downloaded from **this** tenant's Defender portal (Requirement 2.5); re-download if unsure
Onboarded but stays **Inactive** in the portal	Blocked network egress to MDE service endpoints	Run `mdatp connectivity test`; allow the Microsoft Defender service URLs through the proxy/firewall (Requirement 2.8)

E. User prompts appearing (should have been silent)

- Prompts for extension approval, Full Disk Access, or network filter mean the corresponding profile did not apply before the app. Confirm the profile Succeeded and, on unsupervised Macs, expect one-time approval; re-enroll via ADE for fully silent approval.

F. App will not install

Symptom	Cause	Remediation
`MDE macOS – Defender App` shows failed install	Not enrolled/synced, or packaging/type issue	Confirm enrollment (`MACMDE-Discovery.ps1`) and group membership; force a sync; if using a packaged PKG/DMG, confirm the app type and signing

G. Ordering mistake (onboarding pushed first)

- If Macs were onboarded before the capability profiles, they are likely degraded. **Roll back onboarding** (unassign `MDE macOS – Onboarding`), ensure the five capability profiles Succeed, then reassign onboarding. Do not try to "fix forward" a degraded agent by re-onboarding on top of missing approvals.

H. Graph scripts fail to connect

- The scripts need Microsoft Graph PowerShell and read scopes such as `DeviceManagementConfiguration.Read.All`, `DeviceManagementApps.Read.All`, `DeviceManagementManagedDevices.Read.All`; consent them on first `Connect-MgGraph`.
- If module load fails, run `Install-Module Microsoft.Graph -Scope CurrentUser`.

I. Diagnostics quick reference

```
Get-MgContext   # confirm Graph scopes
```

On the Mac (Terminal — the definitive agent state):

```
mdatp health                # healthy, real_time_protection_enabled, licensed, onboarded
systemextensionsctl list    # Defender endpoint-security + network extensions enabled?
mdatp connectivity test     # reach the MDE cloud?
profiles list               # installed configuration profiles (the five + onboarding)
```

Intune-side: device → **Device configuration** shows each profile's apply status; **Apps** shows app install state; the onboarding **custom profile** shows Succeeded/Error.

J. When to escalate

If all five capability profiles show Succeeded, the app is Installed, `MDE macOS – Onboarding` is Succeeded, `mdatp connectivity test` passes, and the agent still reports unhealthy, capture `mdatp health`, `systemextensionsctl list`, `profiles list`, and the Intune device-status pages and open a Microsoft support case.

7. Best Practices — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune)

- **Profiles first, onboarding second — always.** The single most important practice: deliver and

confirm the five capability profiles (System Extensions → Full Disk Access → Network Filter → Background Services → Notifications) **before** the app and the onboarding package. Onboarding a Mac whose extensions and Full Disk Access are not yet approved yields a silently degraded agent that looks onboarded but protects nothing.

- **Follow the extension-approval order.** System Extensions must be approved so the endpoint-security

and network extensions can load; Full Disk Access (PPPC/TCC) must be granted so the endpoint-security extension can read the disk; the Network Filter must pre-approve the network extension so protection can attach. Each depends on the one before — assign them together, but verify them in this order.

- **Prefer supervised (ADE) Macs.** Automated Device Enrollment lets Intune silently approve system

extensions and network filters with no user interaction; on unsupervised Macs users may face one-time prompts. Enroll via ADE where you can.

- **Health, not the green tick, is the finish line.** A device showing "onboarded" in the portal is

not proof of protection. Gate on `mdatp health` reporting healthy with `real_time_protection_enabled` true and the extensions running (`systemextensionsctl list`).

- **Pilot before you onboard the fleet.** A small `MDE macOS – Pilot Devices` group soaked for several

business days catches a missing approval or a bad onboarding package on a few Macs instead of all of them.

- **Use Microsoft's published identifiers — never invent them.** The Defender team identifier,

extension bundle identifiers, network extension identifier, PPPC code requirement, and the onboarding package are Microsoft-published and tenant-specific. Always take them from the current Microsoft docs and your tenant's Defender-portal download.

- **Use the right macOS surface for each payload.** Prefer the Settings catalog / Templates where the

payload is exposed; use a custom `.mobileconfig` (Templates → Custom) for the onboarding payload and for any capability payload the catalog cannot reach.

- **Name and tag everything consistently.** Use the `MDE macOS – <purpose>` convention (`MDE macOS –

System Extensions, – Onboarding, etc.) and put the tag MDE-macOS` in every profile/app description so the object set is easy to find and audit.

- **Keep the APNs certificate alive.** Apple management — and therefore every MDE profile and the

onboarding payload — stops the moment the Apple MDM Push certificate expires. Track its renewal date and renew with the same Apple ID before expiry.

- **Expand in the same order you deployed.** When adding production, assign the five capability profiles

to the production group and confirm they apply **before** assigning the app and then the onboarding profile. Roll out in waves and re-check posture after each wave.

- **Keep a rollback ready.** Removing the `MDE macOS – Onboarding` assignment offboards cleanly on the

next cycle; use it to fix a bad onboarding or a degraded agent, then reconfirm the capability profiles and reassign onboarding. Never re-onboard on top of missing approvals.

- **Prove protection with evidence.** Use `mdatp health`, `systemextensionsctl list`, the Defender

portal device state, and `MACMDE-Report.ps1` as your audit evidence that each Mac is genuinely onboarded and healthy.

8. Deliverables Summary — Microsoft Defender for Endpoint for macOS (Deployment & Onboarding via Intune)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The MDE-for-macOS deployment model: system extensions, Full Disk Access (PPPC/TCC), network filter, background services, notifications, and the onboarding blob delivered via Intune — and why order matters
2. Requirements Guide	Every prerequisite with Why / Verify / Implement-if-missing (MDE licensing, APNs, macOS enrollment, onboarding package, extension identifiers, pilot), plus the object-naming convention
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps: build the five capability profiles first, gate, deploy the Defender app, deploy the onboarding profile, validate `mdatp` health on a pilot, roll back if needed, expand
5. Testing Guide	Validation procedures with pass/fail gates — capability-before-onboarding gate, agent-health gate, portal-activity and EDR-detection tests
6. Troubleshooting Guide	Field issues and remediation — starting with the classic "onboarded but not protecting" failure
7. Best Practices	Field-proven recommendations, centered on the extension-approval order

PowerShell scripts

Script	What it does
`MACMDE-Discovery.ps1`	Tenant/device MDE-for-macOS readiness: APNs presence, enrolled Macs, macOS version spread, and existing Defender/security macOS profiles to review for conflicts (read-only)
`MACMDE-Validation.ps1`	Per-device check that a Mac has the five capability profiles, the Defender app, and the onboarding profile in a good state via Intune device config/app states (read-only)
`MACMDE-Report.ps1`	Fleet MDE-for-macOS posture + drift via Microsoft Graph across the five profiles, the app, and the onboarding profile (read-only)
`MACMDE-Remediation.ps1`	Reapplies/repairs drifted MDE objects by re-syncing the affected Macs and confirming assignment; `-WhatIf` by default
`MACMDE-Set-ScopeTargeting.ps1`	Safely widens the target group membership during Expand; `-WhatIf` by default

Object naming convention (used across docs and scripts)

- System Extensions profile: `MDE macOS – System Extensions`
- Full Disk Access (PPPC/TCC) profile: `MDE macOS – Full Disk Access`
- Network filter profile: `MDE macOS – Network Filter`
- Background services profile: `MDE macOS – Background Services`
- Notifications profile: `MDE macOS – Notifications`
- Onboarding profile (custom `.mobileconfig`): `MDE macOS – Onboarding`
- Microsoft Defender app: `MDE macOS – Defender App`
- Device groups: `MDE macOS – Pilot Devices` / `MDE macOS – Production Devices`
- Tag in every profile/app description: `MDE-macOS`

What the buyer receives

- These guides as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable, order-safe method: deliver the capability profiles first, then the app and onboarding, then prove health — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Every Mac onboarded to Microsoft Defender for Endpoint **correctly** — the required system-extension, Full Disk Access, network-filter, background-service, and notification approvals in place *before* the onboarding blob — with the agent proven healthy (extensions running, real-time protection on, active in the Defender portal), validated on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MACMDE-Discovery.ps1

Purpose: Reports whether the tenant and its Macs are ready to deploy and onboard Microsoft Defender for Endpoint on macOS: that an Apple MDM Push (APNs) certificate exists and is not near expiry, how many managed Macs are enrolled, the macOS version spread, and whether any existing macOS configuration profiles already deliver Defender/security payloads (potential conflicts). Read-only against Microsoft Graph (Intune). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACMDE-Discovery.ps1 Requires the Microsoft Graph SDK and these read scopes (consented on first sign-in): DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All, DeviceManagementApps.Read.All. EXPECTED RESULTS APNs certificate presence/expiry, count of enrolled Macs, a macOS-version breakdown, and a list of existing macOS configuration profiles / Defender apps to review before you build. PASS when APNs is present and Macs are enrolled on a supported macOS version. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\MACMDE-Discovery.ps1 Requires the Microsoft Graph SDK and these read scopes (consented on first sign-in): DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All, DeviceManagementApps.Read.All. EXPECTED RESULTS APNs certificate presence/expiry, count of enrolled Macs, a macOS-version breakdown, and a list of existing macOS configuration profiles / Defender apps to review before you build. PASS when APNs is present and Macs are enrolled on a supported macOS version. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

MACMDE-Remediation.ps1

Purpose: When MACMDE-Validation / MACMDE-Report shows drift, this makes the Intune-managed MDE macOS objects re-apply to the affected Macs: it issues a device sync action (so the capability profiles, the Defender app, and the onboarding profile re-evaluate) and confirms each drifted Mac is still in the assigned MDE macOS group. It does NOT hand-edit the Mac and does NOT push onboarding ahead of the capability profiles — Intune remains the single source of truth. Use this to recover drift, not to force an out-of-order onboarding. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\MACMDE-Remediation.ps1 -NameLike "MDE macOS" Apply: powershell -ExecutionPolicy Bypass -File .\MACMDE-Remediation.ps1 -NameLike "MDE macOS" -WhatIf:$false Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.PrivilegedOperations.All (for the sync action). EXPECTED RESULTS With -WhatIf (default): lists the drifted Macs and the sync actions it WOULD issue. With -WhatIf:$false: issues a sync to each drifted Mac so the MDE macOS objects re-apply. Re-run MACMDE-Validation.ps1 after a policy cycle and confirm 'mdatp health' on the device. .NOTES White-label — the repeatable way. Defaults to -WhatIf (no change unless -WhatIf:$false). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$NameLike = 'MDE macOS')

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\MACMDE-Remediation.ps1 -NameLike "MDE macOS" Apply: powershell -ExecutionPolicy Bypass -File .\MACMDE-Remediation.ps1

-NameLike "MDE macOS" -WhatIf:$false Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.PrivilegedOperations.All (for the sync action). EXPECTED RESULTS With -WhatIf (default): lists the drifted Macs and the sync actions it WOULD issue. With -WhatIf:$false: issues a sync to each drifted Mac so the MDE macOS objects re-apply. Re-run MACMDE-Validation.ps1 after a policy cycle and confirm 'mdatp health' on the device. .NOTES White-label — the repeatable way. Defaults to -WhatIf (no change unless -WhatIf:$false). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$NameLike = 'MDE macOS')

MACMDE-Report.ps1

Purpose: Reports, across the Mac fleet, how each device evaluates against the MDE-for-macOS object set in Intune — the five capability profiles (System Extensions, Full Disk Access, Network Filter, Background Services, Notifications), the Microsoft Defender app install state, and the onboarding profile — by reading each object's device statuses. Summarizes succeeded vs drifted (error/conflict/failed/notApplicable) per object so you can spot Macs onboarded without their capability profiles. Read-only against Graph. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACMDE-Report.ps1 -NameLike "MDE macOS" -CsvPath C:\Temp\macmde-posture.csv Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All, DeviceManagementApps.Read.All. EXPECTED RESULTS A per-object breakdown of device states (succeeded/error/conflict/failed/etc.) and an optional per-device CSV showing which Macs have drifted. PASS when capability profiles succeed and no Mac is onboarded ahead of its profiles. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$NameLike = 'MDE macOS', [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\MACMDE-Report.ps1 -NameLike "MDE macOS" -CsvPath C:\Temp\macmde-posture.csv Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All, DeviceManagementApps.Read.All. EXPECTED RESULTS A per-object breakdown of device states (succeeded/error/conflict/failed/etc.) and an optional per-device CSV showing which Macs have drifted. PASS when capability profiles succeed and no Mac is onboarded ahead of its profiles. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$NameLike = 'MDE macOS', [string]$CsvPath)

MACMDE-Set-ScopeTargeting.ps1

Purpose: Connects to Microsoft Graph and adds members (Macs or groups) to an MDE macOS target group during the Expand phase — for example adding a business-unit device group to 'MDE macOS – Production Devices' so more Macs receive the capability profiles, the Defender app, and onboarding. Runs in preview mode by default (-WhatIf) and only changes membership when you pass -WhatIf:$false. Placeholders use hyphenated names such as your-tenant-id and your-member-object-id. Expand the CAPABILITY profiles first, then app, then onboarding. HOW TO RUN Preview (no change): powershell -ExecutionPolicy Bypass -File .\MACMDE-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id Apply: powershell -ExecutionPolicy Bypass -File .\MACMDE-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id -WhatIf:$false Scopes requested: Group.ReadWrite.All, Directory.Read.All EXPECTED RESULTS Preview lists the members that would be added and changes nothing. Apply adds them and confirms. PASS when the target group contains the intended members afterward. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph except the explicit membership add performed only when -WhatIf:$false is passed. #>

How to run: Preview (no change): powershell -ExecutionPolicy Bypass -File .\MACMDE-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id Apply: powershell -ExecutionPolicy Bypass -File .\MACMDE-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id -WhatIf:$false Scopes requested: Group.ReadWrite.All, Directory.Read.All EXPECTED RESULTS Preview lists the members that would be added and changes nothing. Apply adds them and confirms. PASS when the target group contains the intended members afterward. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph except the explicit membership add performed only when -WhatIf:$false is passed. #>

MACMDE-Validation.ps1

Purpose: Checks one Mac's Intune state for the MDE-for-macOS object set and confirms the correct ORDER: that the five capability profiles (System Extensions, Full Disk Access, Network Filter, Background Services, Notifications) are in a good state, the Microsoft Defender app is installed, and the onboarding profile is applied. Flags the dangerous case where onboarding is applied while a capability profile is not. Read-only — it does not change anything. (The definitive agent health — extensions running, real-time protection on — is read on the Mac itself with 'mdatp health'; this script proves the Intune management prerequisites.) HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACMDE-Validation.ps1 -DeviceName MAC-001 Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementApps.Read.All. EXPECTED RESULTS A per-object PASS/DRIFT table for that Mac and 'DEVICE READY' when the five profiles pass, the app is installed, and onboarding is applied. Warns if onboarding is applied before the capability profiles are healthy. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$DeviceName, [string]$NameLike = 'MDE macOS')

How to run: powershell -ExecutionPolicy Bypass -File .\MACMDE-Validation.ps1 -DeviceName MAC-001 Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementApps.Read.All. EXPECTED RESULTS A per-object PASS/DRIFT table for that Mac and 'DEVICE READY' when the five profiles pass, the app is installed, and onboarding is applied. Warns if onboarding is applied before the capability profiles are healthy. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$DeviceName, [string]$NameLike = 'MDE macOS')